This creates a lot of data, over 5 million rows to cover that time period in seconds. The rows are considered to have a "high correlation" because the values for "created_at" are clustered together with how they're stored on disk and they're similar in time. Low correlation would be values that are similar in time but scattered all over.

This high correlation meets the guideline above for a BRIN index to be effective.

From psql, run this SQL statement:

sql/trip_positions_populate_correlation.sql

```
INSERT INTO trip_positions (
  position, trip_id, created_at, updated_at)
  SELECT POINT('(37.769233' ||
      FLOOR(RANDOM() * 10 + 1)::TEXT ||
      ',-122.3890705)'),
  (SELECT MIN(id) FROM trips),
  seq,
  seq
  FROM generate_series(
    '2023-08-01 0:00'::timestamptz,
    '2023-10-01 0:00'::timestamptz,
    '1 second'::interval)
  AS t(seq);
```

With that data created, you're ready to query it. Run the following query in psql to get counts by day, grouping hundreds of thousands of records by day from the first 5 days of the month.

sql/trip_positions_by_day.sql

```
SELECT
  date_part('day', created_at) AS date,
  COUNT(*)
FROM trip_positions
WHERE created_at >= '2023-08-01' AND created_at < '2023-08-06'
GROUP BY 1
ORDER BY 1;
```

Prepend EXPLAIN ANALYZE and confirm that the query uses a Sequential Scan of the trip_positions table.

The WHERE clause above can use an index on the created_at column. To improve the speed of WHERE clause filtering, use a B-Tree or BRIN index. As you saw in the earlier posts, the B-Tree and BRIN indexes may offer comparable performance but the BRIN index will be much smaller.

To create the BRIN index, use the USING brin clause. Run this statement from psql to add the index and then run ANALYZE trip_positions: